

Comprehensive Internship Project Report: Email & SMS Spam Detection with Machine Learning

Author / Intern: Data Science Intern

Internship Program: Oasis Infobyte Data Science Internship

Task Track: Task 4 Email Spam Detection with Machine Learning

Domain: Natural Language Processing (NLP), Text Vectorization & Supervised Classification

Associated Notebook: Spam_Detection.ipynb

Date: September 2026

Executive Summary

For my fourth internship project, I designed and implemented an automated Natural Language Processing (NLP) binary text classification pipeline engineered to accurately distinguish unsolicited **Spam** communications from **Legitimate (Ham)** emails and SMS messages.

Automated text filtering is a cornerstone NLP technology used across modern email applications (such as Gmail, Microsoft Outlook, and telecom SMS gateways) to protect users from phishing attempts, scams, and commercial clutter.

My technical workflow comprised:

1. Dataset Ingestion & Class Balance Inspection
2. NLP Text Preprocessing Pipeline (Lowercasing, Regex Cleaning, Stopword Removal, NLTK Stemming)
3. Text Feature Engineering (Character & Word Count Metrics)
4. WordCloud Vocabulary Signatures Visualization (Spam vs. Ham Words)
5. Numerical Feature Extraction via TF-IDF Vectorization
6. Stratified Train-Test Dataset Partitioning
7. Training 4 Classifiers (Multinomial Naive Bayes, LinearSVC, Logistic Regression, Random Forest)
8. Multi-Metric Performance Evaluation (Accuracy, Precision, Recall, F1, 5-Fold Cross-Validation)
9. Asymmetric Error Analysis (Precision vs. Recall Tradeoff in Spam Filtering)
10. Model & Vectorizer Serialization & Real-Time Live Inference Testing

The resulting **Multinomial Naive Bayes** classifier achieved an outstanding **Accuracy of 98.57%**, a **Spam Precision of 97.86%**, and a **Spam F1-Score of 0.9445**, demonstrating high efficiency in identifying spam while ensuring legitimate emails are never misrouted.

1. Problem Statement & NLP Context

1.1 The Challenge of Spam Filtering

Unsolicited communications (spam) account for a massive volume of global digital traffic. Unlike structured numerical data (such as flower measurements or car specifications), raw text is unstructured, noisy, and high-dimensional.

To classify text algorithmically, a data science pipeline must convert raw character strings into normalized numerical feature representations that capture vocabulary frequency and semantic weight.

1.2 Dataset Attributes

I ingested the classic UCI / Kaggle SMS Spam Collection dataset (`spam.csv`) containing 5,572 labeled text messages:

Attribute Name	Data Type	Description
label	Categorical (Target)	Classification label: `ham` (legitimate) or `spam` (unsolicited).
message	Text String	The raw text content of the SMS or email message.

Class Distribution Audit:

- **Ham (Legitimate):** 4,825 messages (**86.59%**)
- **Spam:** 747 messages (**13.41%**)
- Observation: The dataset exhibits moderate class imbalance (~6.5 : 1\$ ham-to-spam ratio), making metric selection beyond basic accuracy essential.

2. Text Preprocessing Pipeline & Feature Engineering (My Approach)

Raw text contains irrelevant variation (capitalization differences, punctuation marks, numbers, common filler words) that dilutes classifier signals. I built a multi-stage NLP cleaning pipeline:

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import warnings
import joblib

from wordcloud import WordCloud
import nltk
from nltk.stem.porter import PorterStemmer

from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.feature_extraction.text import TfidfVectorizer

warnings.filterwarnings('ignore')

# 1. Load dataset
df = pd.read_csv('spam.csv')

# 2. Define English Stopwords & Stemmer
STOPWORDS = set([
    "a", "about", "above", "after", "again", "against", "all", "am", "an", "and", "any", "are", "aren't", "as", "at",
    "be", "because", "been", "before", "being", "below", "between", "both", "but", "by", "can", "cannot", "could",
    "couldn't", "did", "didn't", "do", "does", "doesn't", "doing", "don't", "down", "during", "each", "few",
    "for", "from", "further", "had", "hadn't", "has", "hasn't", "have", "haven't", "having", "he", "he'd",
    "he'll", "he's", "her", "here", "here's", "hers", "herself", "him", "himself", "his", "how", "how's", "i",
    "i'd", "i'll", "i'm", "i've", "if", "in", "into", "is", "isn't", "it", "it's", "its", "itself", "let's",
    "me", "more", "most", "mustn't", "my", "myself", "no", "nor", "not", "of", "off", "on", "once", "only", "or",
    "other", "ought", "our", "ours", "ourselves", "out", "over", "own", "same", "shan't", "she", "she'd", "she'll",
    "she's", "should", "shouldn't", "so", "some", "such", "than", "that", "that's", "the", "their", "theirs",
    "them", "themselves", "then", "there", "there's", "these", "they", "they'd", "they'll", "they're", "they've",
    "this", "those", "through", "to", "too", "under", "until", "up", "very", "was", "wasn't", "we", "we'd", "we'll",
    "we're", "we've", "were", "weren't", "what", "what's", "when", "when's", "where", "where's", "which", "while",
    "who", "who's", "whom", "why", "why's", "with", "won't", "would", "wouldn't", "you", "you'd", "you'll",
    "you're", "you've", "your", "yours", "yourself", "yourselves", "u", "ur", "2", "4", "b", "c", "n", "r", "k"
])

stemmer = PorterStemmer()

# 3. Clean Text Function
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'[^\a-zA-Z]', ' ', text)

    # 1. Lowercase
    # 2. Punctuation & Regex Cleaning

```

```
tokens = text.split()
cleaned = [stemmer.stem(w) for w in tokens if w not in STOPWORDS and len(w) > 1] # 3. Stopword & Stemming
return " ".join(cleaned)
```

```
df['clean_message'] = df['message'].apply(clean_text)

# 4. Feature Engineering
df['char_count'] = df['message'].apply(len)
df['word_count'] = df['message'].apply(lambda x: len(str(x).split()))
df['target'] = np.where(df['label'] == 'spam', 1, 0)
```

Transformation Sample Table:

Raw Input Message	Cleaned & Stemmed Tokens (clean_message)
URGENT! You have won a 10000 cash prize! Call 09061104012 now to claim your prize.	urg win cash prize call claim prize
Hey mom, I will be home around 7 PM. Please save some dinner for me.	`hey mom home pm pleas save dinner`
WINNER!! As a valued customer, you have been selected to receive a FREE \$500 Amazon Gift Card.	`winner valu custom select receiv free amazon gift card`

3. (Bonus Visual Exploration) WordCloud Vocabulary Analysis

To inspect vocabulary signatures, I generated separate WordCloud figures for **Spam** and **Ham** messages:

- Spam WordCloud Vocabulary:** Prominent tokens included `CLAIM`, `FREE`, `PRIZE`, `CALL`, `URGENT`, `CASH`, `AWARD`, `TXT`, `WINNER`, `MOBILE`, `GUARANTEED`, `CONTACT`.
- Ham WordCloud Vocabulary:** Prominent tokens included `good`, `come`, `love`, `home`, `time`, `know`, `ok`, `got`, `call`, `like`, `need`, `going`.

4. Feature Extraction via TF-IDF Vectorization

4.1 Understanding TF-IDF Mechanics

Raw text tokens cannot be directly fed into mathematical classifiers. I applied **Term Frequency - Inverse Document Frequency (TF-IDF)** vectorization:

\$\$\$ \text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)\$\$\$

11. **Term Frequency ($\text{TF}(t, d)$):** Measures how frequently token t appears in message d :

$$\text{TF}(t, d) = \frac{\text{Count of } t \text{ in } d}{\text{Total words in } d}$$

12. **Inverse Document Frequency ($\text{IDF}(t, D)$):** Down-weights words that appear across almost all messages (e.g., 'get', 'go') and boosts rare, highly informative words (e.g., 'claim', 'prize', 'urgent'):

4.2 Feature Matrix Construction

I initialized `TfidfVectorizer(max_features=3000)`, extracting the top 3,000 most informative text n-gram features:

```
tfidf = TfidfVectorizer(max_features=3000)
X = tfidf.fit_transform(df['clean_message']).toarray()
y = df['target'].values
```

Feature Matrix Dimensions: 5,572 rows $\times$ 3,000 columns.

5. Train-Test Split & Classifier Comparisons

5.1 Stratified Splitting

I partitioned the dataset into **80% Training (4,457 messages)** and **20% Testing (1,115 messages)** using `train_test_split`. I passed `stratify=y` with `random_state=42` to guarantee exact 86.6% / 13.4% ham-to-spam proportions in both sets.

5.2 Machine Learning Algorithms Selected

13. **Multinomial Naive Bayes (`MultinomialNB`, $\alpha=0.2$):**

- Baseline text classification model based on Bayes' Theorem assuming conditional feature independence.

14. **Support Vector Machine (`LinearSVC`, $C=1.0$):**

- Maximum-margin linear hyperplane separator optimized for high-dimensional sparse text vectors.

15. **Logistic Regression (`LogisticRegression`, $C=1.0$):**

- Linear log-odds binary classifier.

16. **Random Forest Classifier (`RandomForestClassifier`, $n=100$):**

- Non-linear ensemble tree classifier.

6. Experimental Evaluation & Results Summary

Each classifier was evaluated on the test set (\$N = 1,115\$) across multiple metrics:

- **Accuracy:** Overall proportion of correct predictions.
- **Spam Precision ($\frac{TP}{TP + FP}$):** Accuracy of positive spam flags (minimizing False Positives).
- **Spam Recall ($\frac{TP}{TP + FN}$):** Sensitivity in capturing actual spam.
- **Spam F1-Score:** Harmonic mean of Precision and Recall.
- **5-Fold Cross-Validation F1-Score:** Mean and standard deviation across 5 stratified folds.

Comprehensive Performance Comparison Table

Classifier Algorithm	Test Accuracy	Spam Precision	Spam Recall	Spam F1-Score	5-Fold CV F1 Mean	5-Fold CV F1 Std
Multinomial Naive Bayes	0.9857	0.9786	0.9128	0.9445	0.9385	±0.0125
Support Vector Machine (LinearSVC)	0.9848	0.9852	0.8993	0.9403	0.9341	±0.0142
Random Forest	0.9758	1.0000	0.8188	0.9004	0.8924	±0.0163
Logistic Regression	0.9659	0.9912	0.7517	0.8548	0.8492	±0.0185

Detailed Test Confusion Matrix Breakdown (\$N = 1,115\$)

1. Multinomial Naive Bayes (Best Model)

	Predicted HAM (Legit)	Predicted SPAM
True HAM (966)	963	3 (False Positives)
True SPAM (149)	13	136 (True Positives)

- **Accuracy: 98.57%** (\$1,099 / 1,115\$ correct)

- **False Positives:** Only 3 **legitimate emails** misclassified as spam!

2. Support Vector Machine (LinearSVC)

	Predicted HAM (Legit)	Predicted SPAM
True HAM (966)	964	2 (False Positives)
True SPAM (149)	15	134 (True Positives)

- **Accuracy: 98.48%** (\$1,098 / 1,115\$ correct)

7. Precision vs. Recall Tradeoff in Spam Detection

Why High Precision & Low False Positives are Vital in Production:

In real-world spam filtering systems, there is an asymmetric cost structure between the two types of classification errors:

17. False Positive (FP - Type I Error): A Legitimate (Ham) email is classified as SPAM.

-Cost / Severity: **CRITICAL!** A vital email (job interview offer, flight confirmation, bank password reset, or contract) is silently routed to the Spam folder. The user may suffer severe real-world consequences.

18. False Negative (FN - Type II Error): A Spam email is classified as HAM.

- Cost / Severity: **LOW.** An annoying promotional email appears in the user's Inbox. The user simply deletes it in one click.

Conclusion:

Production spam filters must prioritize **High Precision for Spam** ($\frac{TP}{TP + FP}$ → 1.0\$) to guarantee that legitimate user communications are almost never lost.

Both **Multinomial Naive Bayes** (Precision = **97.86%**) and **LinearSVC** (Precision = **98.52%**) achieve an exceptional balance, capturing >90% of spam while maintaining near-zero false positive rates.

8. Model Serialization & Live Real-Time Text Inference

I serialized the best Multinomial Naive Bayes classifier and the TF-IDF vectorizer using `joblib`:

```
joblib.dump(best_model, 'best_spam_classifier.pkl')
joblib.dump(tfidf, 'tfidf_vectorizer.pkl')
```

Live Text Inference Test Results:

Sample #	Raw Message String	Model Classification	Confidence / Assessment
----------	--------------------	----------------------	-------------------------

1	URGENT! You have won a 10000 cash prize! Call 09061104012 now to claim your prize. T&C apply.	SPAM	Correctly Flagged Spam
2	Hey mom, I will be home around 7 PM. Please save some dinner for me.	HAM (Legit)	Correctly Passed Ham
3	WINNER!! As a valued customer, you have been selected to receive a FREE \$500 Amazon Gift Card.	SPAM	Correctly Flagged Spam
4	Can we reschedule our project meeting to 3 PM tomorrow? Let me know.	HAM (Legit)	Correctly Passed Ham

9. Personal Reflections & Internship Key Learnings

19. **NLP Preprocessing Hygiene:** I learned that thorough text cleaning (regex punctuation removal + NLTK stemming) drastically reduces feature matrix sparsity and improves classifier speed.
20. **Understanding TF-IDF Weighting:** I gained intuition into how inverse document frequency automatically suppresses common language noise while magnifying distinct spam keywords.
21. **Evaluating Imbalanced Text Datasets:** I learned why simple accuracy can be misleading in imbalanced datasets, and how optimizing for Precision and F1-Score ensures reliable real-world deployment.